In this homework, you will use the Spark Streaming API to devise a program which processes a stream of items and compares the effectiveness of two methods to compute the approximate frequent items: (1) sticky sampling; and (2) a method based on count-min sketch.

Description of the methods

Consider a stream of n items, and let $\phi \in (0, 1)$ be a frequency threshold. A **true frequent item** is an item that occurs *at least* ϕn times in the stream. Let $\epsilon \in (0, \phi)$ and $\delta \in (0, 1)$ be suitable accuracy and confidence parameters (for Stricky Sampling), and let d and w the number of rows and columns, respectively, of a Count-Min sketch. We call

- F_{SS} the set of approximate frequent items computed by Sticky Sampling with parameters ϕ, ϵ, δ (i.e., using a sampling rate $r = \ln(1/(\delta\phi))/\epsilon$; and
- F_{CM} the set of frequent items computed by a Count-Min sketch $d \times w$ as follows: an item x is added to F_{CM} the first time its current estimated frequency becomes $\geq \phi n$.

Generation of probabilitites and hash functions

Sticky Sampling adds each new item to the sample with probability $p = r/n$. To this purpose you can use the random generators by provided by Java and Python: generate a random number in $x \in [0, 1]$ and perform the action only if $x \leq p$.

For the Count-min sketch, we advise you to use the following 2-universal family of hash functions, which map arbitrary integers (i.e., the items) into integers in the range $[0, C - 1]$, for some suitably-defined value C. One such function h maps an item x into the value $((ax + b) \bmod p) \bmod C$, where $p = 8191$, a is a random integer in $[1, p - 1]$ and b is a random integer in $[0, p - 1]$. **Important:** note that a **cannot be 0**. In order to define a hash function from this family, for a given C, you must simply generate the two random values $a \in [1, p - 1]$ and $b \in [0, p - 1]$.

Spark streaming setting that will be used for the homework

For the homework, we created a server which generates a stream of **integer items**. The server has been already activated on the machine **algo.dei.unipd.it** and emits the items (viewed as strings) on specific **ports (from 8886 or 8889)**. Your program must first define a **Spark Streaming Context sc** that provides access to the stream through the method **socketTextStream** which transforms the input stream, coming from the specified machine and port number, into a *Discretized Stream* (**DStream**) of **batches of items**. A batch consists of the items arrived during a time interval whose duration is specified at the creation of the context sc. **Each batch is viewed as an RDD of strings**, and a set of RDD methods are available to process it. A method **foreachRDD** is then invoked to process the batches one after the other. Typically, the processing of a batch entails the update of some data structures stored in the driver's local space (i.e., its working memory) which are needed to perform the required analysis. The beginning/ end of the stream processing will be set by invoking **start/stop** methods from the context sc. Typically, the stop command is invoked after the desired number of items is processed.

The **ports from 8886 to 8889** of **algo.dei.unipd.it** generate four streams of 32-bit integers:

- **8887:** it generates a stream where a few elements are very frequent, while all the remaining are randomly selected in the 32-bit integer domain.
- **8889:** it generates a stream where a few elements are very frequent, some elements are moderately frequent, and all the remaining are randomly selected in the 32-bit integer domain.
- **8886:** it is the "deterministic" version of the stream 8887, meaning that it generates the exact same stream every time you connect to this port. It should be used to test your algorithm.
- **8888:** it is the "deterministic" version of the stream 8889, meaning that it generates the exact same stream every time you connect to this port. It should be used to test your algorithm.

To learn more about Spark Streaming you may refer to the official Spark site. Relevant links are:

- Spark Streaming Programming Guide (full documentation)
- Transformations on Streams (list of transformations applicable to the RDDs in a DStream)

Running the program and template

Your program will be run in local mode on your PC (which you used in Homework 1 to devise and debug your program, before running it on the cluster).

IMPORTANT: **the master should be set to local[*]**

This can be done by calling method setMaster("local[*]") when creating the Spark Configuration (see the DistinctItems example described below).

In order to see a concrete application of the above setting you can download and run the **DistinctItemsExample** program which you can find [illegible] **in Java and Python**) in the same section as this page. The program takes as input the port number (*port*) and an approximate number of items to process (*threshold*). It computes the exact number of distinct items in the batches received *up to, and including, the batch* which contains the

threshold-th item of the stream emitted by **machine algo.dei.unipd.it** at port number *port.* Note that, in this fashion, a bit more that *threshold* items will be processed. The discrepancy depends on the time specified for collecting a batch.

We strongly encourage to use this program as a template for your homework.

WARNING: When executing your programs, if you receive an error message such as ***ERROR ReceiverTracker .....*** do not worry. This is triggered by some temporary connection problem between the stream server and Spark context, but it **has no consequence on the correctness of the execution.**

Task for HW2.

You must write a program **GxxHW2.java** (for Java users) or **GxxHW2.py** (for Python users), where xx is your 2-digit group number (e.g., 04 or 45), which receives in input the following 7 **command-line arguments (in the given order):**

- **An integer** *n*: the number of items of the stream to be processed.
- **A float** *phi*: the frequency thresold in $(0, 1)$
- **A float** *epsilon*: the accuracy parameter $(0, \mathrm{phi})$
- **A float** *delta*: the confidence parameter in $(0, 1)$
- **Two integers** *d,w***:** number of rows and columns in the Count-Min sketch
- **An integer** *portExp*: the port number

The program must process the items in the batches received *up to, and including, the batch* which contains the n-th item of the stream emitted by **machine algo.dei.unipd.it** at port number *portExp.* It must compute the following information **relative to the first *n* processed items** (*all items after the n-th one must be ignored*):

- The **true frequent items**, with respect to ϕ
- The set F_{SS}, using parameters ϕ, ϵ, δ
- The set F_{CM} using parameters ϕ, d, w

The program should print:

- The input parameters provided as command-line arguments.
- The true frequent items, in increasing order of item (seen as integer). Print one item per line together with its true frequency.
- The number of items stored in the dictionary (e.g., Hash Table) used by Sticky Sampling
- The items in F_{SS}, in increasing order of item (seen as integer). Print one item per line together with its true frequency.
- The total number of items returned by Count-Min sketch (i.e., the size of F_{CM})
- The items in F_{CM}, in increasing order of item (seen as integer). Print one item per line together with its true frequency.

REMARK: for the items of F_{SS} and F_{CM} we ask you to print their "true" frequencies, which are useful to assess how far the frequencies of the false positives are from the threshold ϕn.

IMPORTANT REQUIREMENTS:

- **Make sure that your program strictly adheres to the output format used in the example** provided in the same section as this page. Any deviation from this format will be penalized.
- **The program that you submit should run without requiring additional files.** Test your program on your local machine using various configurations of parameters.
- **Fill the table given in this word form, reporting the required results.**

USE OF GENERATIVE AI TOOLS. The use of such tools is not forbidden, but must be explicitly declared in the word form to be returned with the code. In all cases, each student must be fully aware of the code submitted by his/her group, and is responsible for the code's correctness and efficiency. We recall that the questions about the homeworks can be asked in the exams.

Submission Instructions

Each group must submit a zipped folder GxxHW2.zip, where xx is your group number. The folder must contain the program (**GxxHW2.java or GxxHW2.py**) **and a pdf file GxxHW2form.pdf** (suitably renaming xx with your group number), which is the pdf version of the word document with the aforementioned table. Only one student per group must do the submission using the link provided in the Homework 2 section. Make sure that your code is free from compiling/run-time errors and complies with the important requirements listed above.

If you have questions about the assignment, contact the teaching assistants (TAs) by email to bdc-course@dei.unipd.it . The subject of the email must be "HW2 - Group xx", where xx is your group number. If needed, a zoom meeting between the TAs and the group will be organized.

Last modified: Saturday, 6 June 2026, 2:01 PM

?